

OpenViking

系统设计与整体流程分析

面向 AI Agent 的上下文基础设施：以可寻址的目录组织资料、记忆和技能，再通过检索、会话归档与知识整理形成可持续复用的上下文闭环。

分析基线	范围与方法
源码版本	55c34934e7b61a8755303e0df1da7a4d4cffcc53
版本核对	2026-10-08 核对时，上游 volcengine/OpenViking 与目标 7774596/OpenViking 的 main 均指向该提交。
分析范围	系统边界、信息模型、组件职责、端到端流程、权限、一致性与设计取舍。
不涵盖	性能参数调优、吞吐基准、完整部署教程；不将设计文档中的承诺自动视为已实现。

阅读导航

页码	主题
02-04	系统边界、统一上下文模型、身份与权限
05-08	资源导入、检索组装、会话归档、长期记忆
09-11	技能与编译、宿主插件与网关、存储一致性
12-14	配置与运行管理、端到端示例、设计判断与版本校准
15-16	源码与文档索引：可点击定位到固定提交

方法：结合当前提交的源码与概念文档进行静态分析。图示为职责和数据流示意，并非逐函数调用图。正文 [Sxx] 为固定提交引用；明确标注的“设计判断”是分析推论。

系统边界：上下文服务与 Agent 分工

核心服务管理“Agent 可以使用什么上下文，以及这些上下文如何更新”。任务决策、工具执行和用户对话通常仍由接入的 Agent 宿主负责。

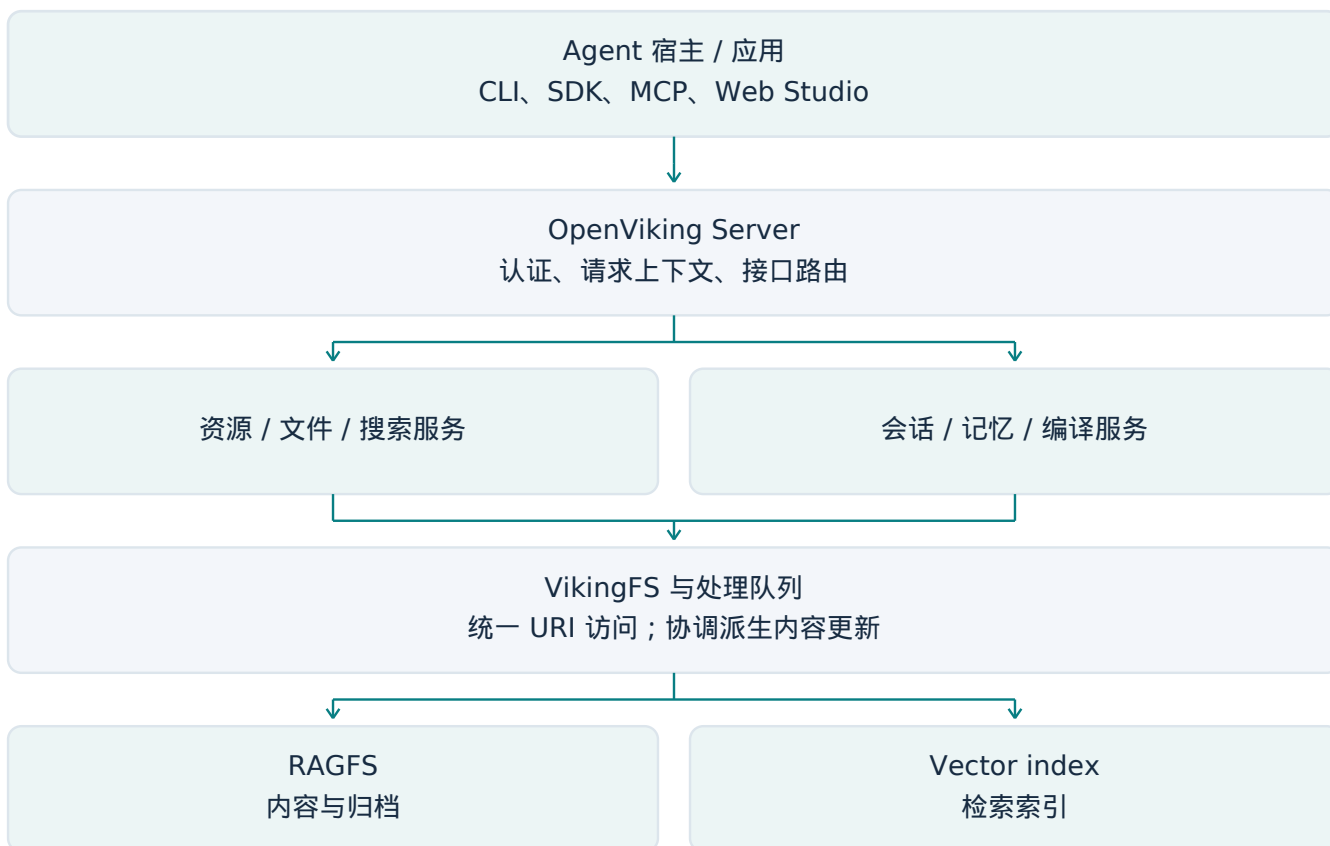


图 1 主服务职责图。异步队列是工作执行机制，不是所有读取请求的必经路径。

OpenVikingService 负责组装业务服务、模型解析器、存储、任务队列及生命周期。HTTP / MCP 是接入表面，Service 层组织业务，解析、检索和记忆模块实现具体流程。[S01][S30]

VikingBot / Agent Runtime 是可选执行侧，可承担上下文编译等 Agent 任务；Context Gateway 则是独立进程，代理模型 API 并接入记忆。二者都不应与核心上下文数据库混为一个组件。[S16][S18]

设计判断：明确上下文服务与宿主的分工，有利于同一份知识被多个客户端复用；这并不意味着任何集成都必须由 OpenViking 接管宿主历史。

统一模型：类型、地址与信息层级

Resource、Memory、Skill 共享目录和 URI 抽象，但各自保留不同的生产方式与更新规则。统一的是访问模型，不是把所有数据强制变成同一种语义。

对象	意义	典型来源 / 使用
Resource	外部知识与参考材料	文档、代码、网页和多媒体导入；检索后读取原文。
Memory	可跨会话复用的信息	从交互中提取画像、偏好、实体、事件及经验；按 Schema 更新。
Skill	可复用的操作方法	SKILL.md 与相关文件构成技能包；可被宿主加载或用于编译。
Session	交互过程的记录	消息、上下文引用、工具记录与归档；是记忆提取的输入。

URI 是稳定地址，目录提供可浏览的组织结构

viking://resources/... 表示账号内共享资料；viking://~/memories、viking://~/skills 表示当前用户空间。~ 按调用者身份解析。用户可通过 ls / tree / read / grep 访问，也可通过语义搜索定位。[S02][S04]

层级	表示	设计作用
L0	.abstract.md	目录简要摘要，用于初步相关性判断。
L1	.overview.md	目录概览与导航，帮助理解范围和选择细节。
L2	完整文件及子目录	保存详细知识、证据与可进一步处理的材料。

L0 / L1 是目录级语义文件，不是每个普通文件都各带两份摘要；两者也不保证始终同时存在。它们由内容派生，带有来源、生成及新鲜度等元数据，不能替代源文件作为证据。[S03]

设计判断：层级同时支持人和 Agent 逐步理解资料。当前检索已采用全局召回，信息分层并不等于检索算法必须逐层遍历。

身份与权限：先确定“谁能看什么”

同一个 `viking://` 地址会在请求身份下解释。URI 本身不包含全部安全边界；`account`、`user`、`peer` 和 `ACL` 共同决定有效访问范围。

边界	职责
Account	最外层租户边界。内容实际存储路径及向量检索范围都绑定账号。
User	隔离用户记忆、私人资源、技能与会话；共享资源由账号范围和权限决定。
Peer	用户空间内与某个交互对象或项目相关的内容范围；不会创建新的租户身份。
Role / ACL	角色负责管理权限；资源 ACL 细化账号内共享资源的授权与继承。



图 2 读文件与搜索共享身份约束，不能把向量搜索视为绕过访问控制的入口。

API Key 模式由凭据解析有效身份；trusted 模式面向受信上游注入身份。业务请求携带 `RequestContext`，下游异步任务也需要保留相应身份，避免后台处理越过用户边界。[S22]

当前 ACL 规则：共享资源根 `viking://resources` 固定授予当前 `account` 的 `user:*` 管理权，子节点默认继承；`restricted` 节点可截断继承。ACL 总开关默认关闭，个人私有区不因此变为共享。[S23]

设计判断：共享、私有和 `peer` 相关内容应在建模时分清；目录名字相同，不代表两个账号能够访问同一份内容。

04 / INGESTION

资源导入：把外部材料变成可检索上下文

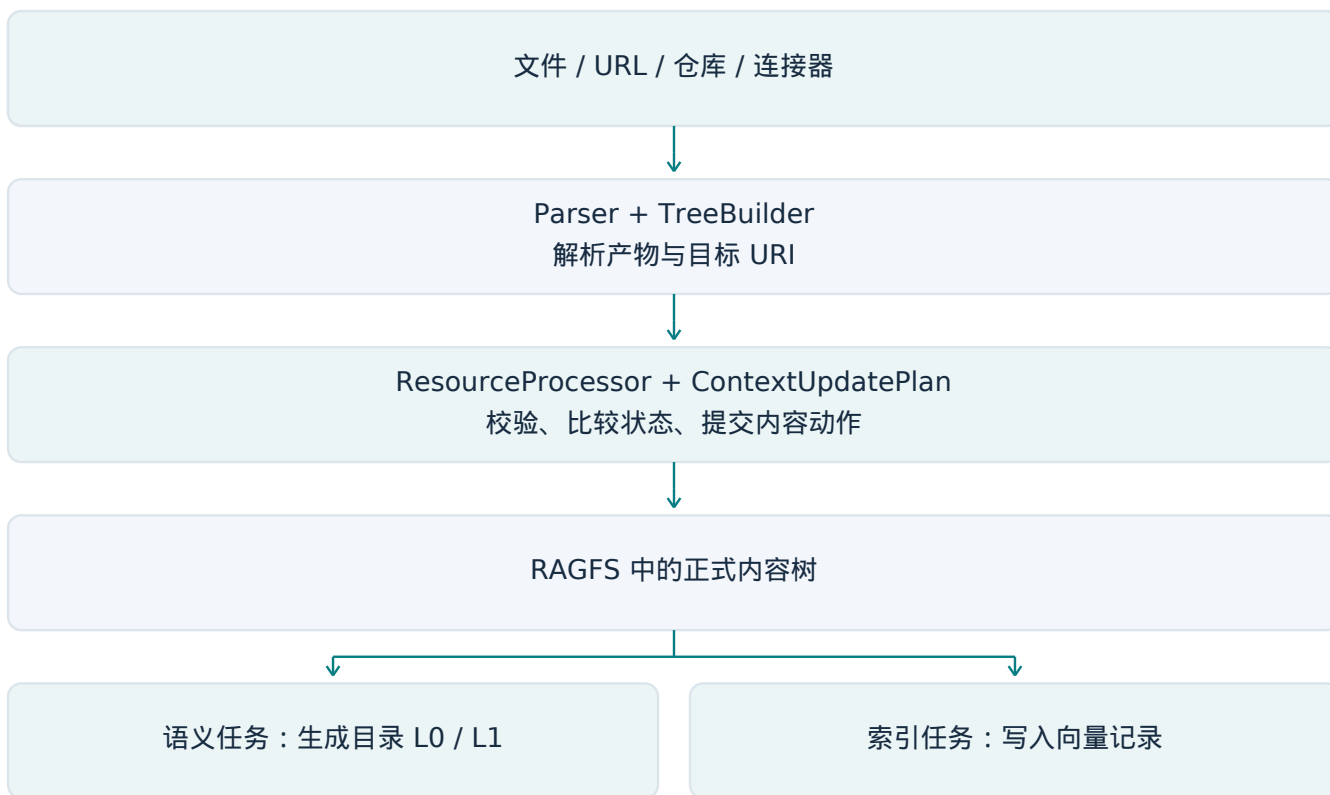


图 3 主导入流程。两类派生任务由计划决定依赖；需要新摘要的索引会在摘要生成之后更新。

先表达目标状态，再执行内容与派生动作

解析器把来源转为文件和目录；具体解析器可以使用模型或外部解析服务。TreeBuilder 确定目标与产物引用，本身不承担完整内容提交。ResourceProcessor 根据请求、解析产物、现有内容和索引编译更新计划。[\[S05\]](#)[\[S06\]](#)

计划中的内容动作先进入正式存储，随后异步完成摘要和向量更新。目录语义从叶子向父级组织：文件摘要与子目录摘要参与目录概览，形成 L1，再提取 L0。这是语义生成的层级关系，并非查询时的逐层搜索。[\[S03\]](#)[\[S06\]](#)

接口接收导入、内容已写入、摘要已更新、索引可检索，是不同状态。需要依赖本次导入执行下一步时，应检查对应任务结果，而不能把返回任务 ID 当作全部处理成功。[\[S24\]](#)

设计判断：统一计划把新增和更新纳入同一套状态转换；内容与派生处理分阶段，也要求显式暴露处理中和失败状态。

检索：从命中列表到可直接注入的上下文

当前版本的核心检索是“在授权范围内全局召回，再可选排序”。目录结构帮助组织和解释结果，但不再驱动目录优先队列式搜索。

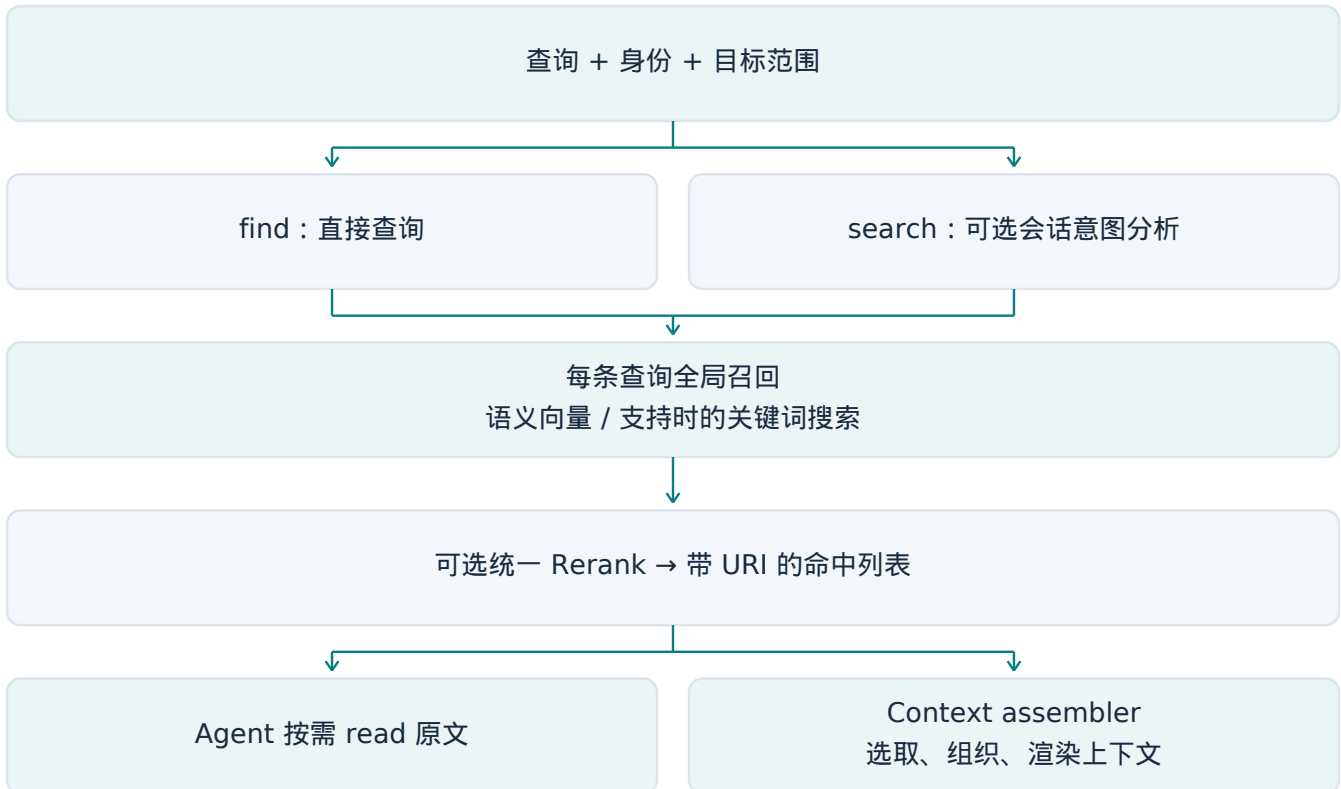


图 4 两种消费形态：返回候选列表，或通过上下文组装管线提供可注入材料。

find 不做会话意图分析；search 可在具备会话内容且开启相关配置时生成类型化查询。

HierarchicalRetriever 的名字仍保留，但实现已是全局搜索，不再做父子分数传播和多轮目录收敛。[S07]

Context assembler 在候选之上处理类别、来源范围、排除项和详情层级，按上下文预算规划内容，必要时读取正文，再输出带 URI 的条目、渲染文本及可选摘要。它承担“给当前任务准备上下文”，而非直接替 Agent 回答问题。[S08]

边界：BM25 keywords 路径需要受支持的远程 VikingDB 全文索引；不能把该能力推定为所有存储后端都具备。检索摘要用于选择，精确事实应回读原文。[S28]

06 / SESSION LIFECYCLE

会话：记录交互、建立归档边界、触发沉淀

Session 是过程记录，长期 Memory 是可复用信息，Working Memory 是用于衔接会话的摘要。三者职责不同，不能用“记忆开关”一概描述。



图 5 会话提交的职责分离。归档保留原始材料，后台任务承担语义提取。

Phase 1 决定哪些消息进入归档、哪些保留在活动会话，并留下可恢复状态。Phase 2 通过持久化 `session_commit` 队列执行；重启后根据归档和队列继续工作。调用者以 `task_id` 跟踪进度。[S09]

版本校准：升级文档称服务端 WM 默认关闭，但该提交 `MemoryPolicy.working_memory_enabled` 及缺省解析仍为 `True`。Codex 插件的 `resumeArchiveInject` 已默认 `False`。两者是不同层的行为；本报告以源码区分，不把文档的新默认值当作服务端现状。[S10][S14][S25]

设计判断：宿主历史管理、归档保存和长期提取应能够独立演进。插件不注入历史，并不等于服务端不生成摘要；实际生成仍取决于生效的记忆策略。

07 / LONG-TERM MEMORY

长期记忆：从交互证据到受约束的知识更新



图 6 记忆写入是受类型约束的更新流程，而不是把聊天摘要直接追加到一个文本文件。

画像、偏好、实体、事件等类型由注册的 Schema 描述，模型负责从上下文识别值得保留的信息。

ExtractLoop 结合既有记忆进行提取；MemoryUpdater 负责把解析后的操作落到具体文件，并维护合并、替换和引用关系。[S11][S12][S13]

self 与 peer 策略决定写入哪个身份空间，memory_types 决定提取范围。类型和目标的约束使“关于用户的稳定偏好”与“某项目的一次事件”能够分别保存，而不是混成一段无边界的历史。[S10][S22]

开启 experiences 相关策略时，cases、trajectories 和 experiences 组成额外的经验演化链路：利用任务过程及结果形成、更新可复用经验或技能策略。这里的演化针对持久化经验与技能内容，不能直接等同于基础模型权重训练。[S10][S11]

设计判断：Schema 限制输出形状，来源和变更记录帮助追溯，但并不自动证明模型提取的事实正确。重要记忆仍需保留回到原始交互验证的能力。

技能与编译：让已有上下文形成新产物

Skill 将“怎样做”作为可管理内容保存。Compile 将资料、技能与执行侧连接起来；记忆整理则走专门的内部更新路径。

能力	输入	结果与执行者
Skill 管理	SKILL.md、辅助文件、来源包	导入、索引、发现、读取；宿主或运行时决定如何执行技能。
普通 Compile	来源 URI + Skill URI + 目标 URI	由配置的 Agent Runtime 执行，生成 Wiki、日报、知识图谱等内容。
Memory 整理	已有 memories 目录 + 整理指令	服务进程内使用记忆框架整理，不经过普通 VikingBot 编译路径。

两条路径具有不同的写入语义

普通 compile 以来源材料为依据，按 Skill 描述的结构生成产物；任务使用请求用户身份访问材料并写入目标，通过任务接口查询结果。这是把数据库中的知识交给执行侧加工，再返回数据库的闭环。[\[S18\]](#)

--skill memory 是特殊路由值，不是真实 Skill URI。它对指定记忆空间就地去重、合并、拆分或精简，不接受外部 --from。改名、替换与关联关系需要随更新一起处理，操作结果包含实际变更文件和错误。[\[S18\]](#)[\[S19\]](#)

边界：检索与读取是观察行为；compile 与记忆整理会写入内容。任务 completed 也不替代结果检查，尤其要区分无变更、部分成功和完整成功。[\[S19\]](#)

设计判断：资料、方法和产物都用同一套 URI 管理，使知识加工可以重复组合；执行者与存储者分开，也便于更换运行时。

接入：宿主插件与上下文网关各司其职

维度	宿主插件 / MCP	Context Gateway
接入位置	宿主生命周期 hook 与工具接口	客户端和模型上游之间的 API 代理
主要行为	自动召回、捕获消息、提交会话；Agent 主动调用工具	增强请求上下文、保存对话，并可代执行记忆工具
适用对象	支持插件、hook 或 MCP 的 Agent	能修改 Base URL 和 API Key 的模型客户端
边界	历史与工具能力受宿主适配影响	独立 Beta 服务；同协议转发，不做协议转换

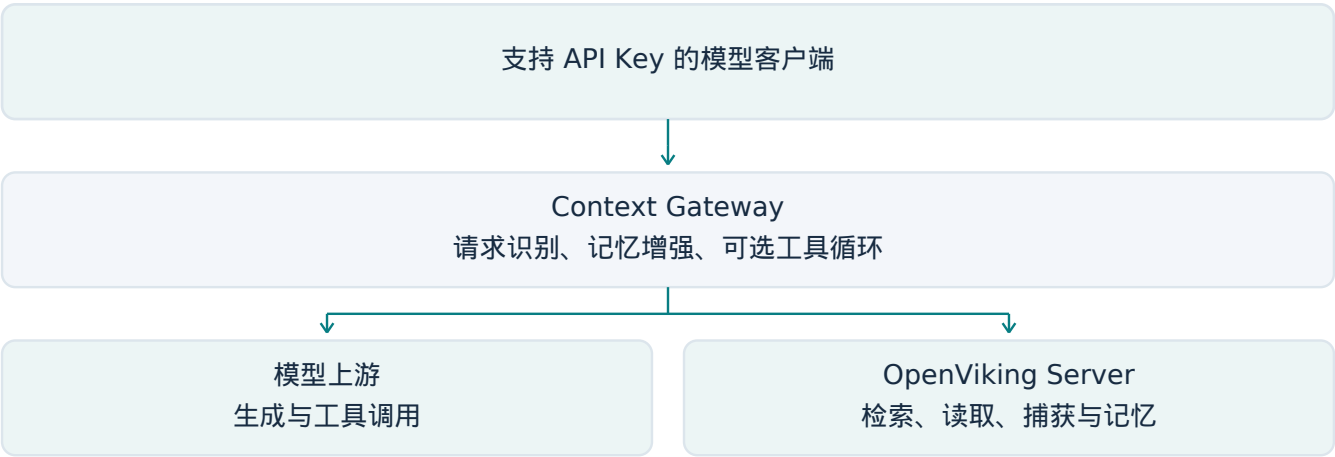


图 7 网关的两类依赖：模型上游负责生成，OpenViking 保存会话与记忆正本。

Codex 插件在用户输入时召回、在交互后捕获，并在相应生命周期提交。显式工具调用可以进一步读取原文。新加入的来源摘要展示本轮提供或读取过的来源；它不能证明答案使用了每一条来源。[\[S15\]](#)[\[S30\]](#)

网关通过 OpenViking 的公开接口处理记忆，支持 Messages、Chat Completions、Responses 等协议；Responses 路径要求请求携带完整历史。直接订阅登录并不等同于此 API Key 接入方式。网关还检测已有插件或 MCP，避免重复介入。[\[S16\]](#)[\[S17\]](#)

设计判断：插件更贴近宿主语义，网关覆盖更多通用客户端。二者是集成策略的选择，核心记忆库无需为每种宿主重新实现。

存储：保留内容正本，维护可重建索引

VikingFS 统一访问与同步动作；RAGFS 保存内容，向量后端保存检索表示。索引含摘要等检索文本，但不取代源内容的权威地位。

层	承担的职责
VikingFS	URI 解析、内容访问、语义层级读取、检索接口及文件变更后的索引协调。
RAGFS	Rust 文件系统实现；承接内容后端、路径锁及可选加密等能力。部分旧接口仍沿用 AGFS 名称。
Vector index	URI、层级、向量、元数据和检索文本；按身份及目标范围召回。
Queue / Task	队列负责工作获取与确认；任务记录负责表达业务执行状态和结果。

一致性靠有序操作与恢复，而非跨组件大事务

内容、摘要和索引并非总在同一时刻更新。路径锁协调冲突写入，计划表达待执行动作，持久化队列支持恢复。它们不构成文件系统、向量库和队列之间的统一原子事务。[\[S04\]](#)[\[S20\]](#)[\[S21\]](#)

例如删除优先清除索引，再删除内容，降低返回失效路径的风险；资源导入则先提交内容，再补齐语义和索引。故障可能留下“内容还在但暂时搜不到”的状态，需要重试或重建收敛。[\[S21\]](#)

资源 / 技能 reindex 根据请求、正式内容和向量现状生成计划，由持久化处理器执行；用于修复派生表示，而非重新导入外部源或回放全部历史记录提取。[\[S29\]](#)

“队列为空”只代表当前没有待确认工作，不证明所有业务成功。判断一次导入或提交是否完成，应检查对应 task 的状态、错误与结果。[\[S24\]](#)

配置与运行管理：贯穿流程的控制能力

系统除了处理内容，还需要明确每个请求使用哪套模型、哪份配置、哪个身份，以及失败后如何解释状态。这些能力贯穿业务流程。

关注点	设计职责
模型依赖	VLM / LLM 负责语义生成与提取；Embedding 负责检索表示；Query planner 与 Rerank 负责可选的查询理解和排序。
配置解析	运行时配置管理将账号配置、有效视图与资源生命周期连接起来；模型和存储资源需要按实际账号解析。
可观测性	任务、队列、检索统计、trace 和使用记录用于区分接收、执行、完成及失败。
数据保护	身份授权限制访问；可选静态加密保护存储内容；技能隐私处理与内容层职责分开。
管理表面	Web Studio、CLI 与 API 提供管理入口，业务规则仍由服务端实施。

模型能力是可替换依赖，业务状态留在系统内

服务在请求身份和有效配置下获取模型资源，而不是把某家供应商硬编码为整个系统。模型返回结果还需经过 Schema、操作解析和写回逻辑，才能形成正式记忆。[\[S01\]](#)[\[S12\]](#)[\[S26\]](#)

部署与故障边界应可解释

客户端经 HTTP / MCP 接入主服务；内容后端、向量后端、模型提供方和可选网关具有不同故障边界。网关访问 OpenViking 失败时可以退化为不带记忆的对话，但网关本身中断会影响经过它的模型请求。[\[S16\]](#)[\[S20\]](#)

静态加密由 RAGFS 包装层处理，授权读取仍返回明文；启用加密不会自动迁移所有旧明文文件。备份和恢复也应同时考虑内容、密钥及相应索引状态。[\[S27\]](#)

设计判断：可观测性要回答“上下文是否被保存、能否检索、被提供给谁”，而不只记录某个模型请求是否返回成功。

一个闭环：项目资料如何变成下一次的帮助

以下是基于组件职责组合的示例，用来连接各条流程；它不是自动执行所有步骤的固定工作流。

阶段	发生什么	系统内的对应变化
1	导入项目资料	Parser 解析代码与文档；更新计划把内容写入资源目录，语义与索引任务补齐可检索表示。
2	用户提出任务	插件或应用携带用户与项目范围发起搜索；全局召回资料、相关记忆与可用技能。
3	准备和验证上下文	Context assembler 提供带来源的材料；Agent 对关键事实 read 原文，再结合宿主中的当前对话完成任务。
4	捕获本次交互	消息、上下文引用与可用工具记录进入 Session；宿主历史仍按宿主自己的机制管理。
5	归档并沉淀	commit 固定归档边界；后台按策略保存偏好、实体、事件或经验，更新对应内容与索引。
6	后续交互复用	下一次相关问题召回这些记忆；需要整理时，显式运行 Compile 或 Memory 整理，形成新产物。

每一步都有不同的“完成”

资料已导入不等于向量已可用；消息已捕获不等于记忆已提取；检索命中不等于内容已读；内容已提供不等于答案必然使用。把这些状态分开，是解释整个系统行为的关键。[\[S05\]](#)[\[S08\]](#)[\[S09\]](#)[\[S15\]](#)[\[S24\]](#)

闭环需要来源与身份一直保留

URI 让结果可以被回读，Session 归档让记忆可追溯，RequestContext 让检索和写回保持在有效空间内。长期记忆服务于新的任务，但不应把一次推断自动升级为不可质疑的事实。[\[S13\]](#)[\[S22\]](#)

设计判断：OpenViking 的完整价值来自这些阶段之间的连接，而不是任意一个独立的向量搜索接口。

设计判断与当前版本的校准点

设计选择	带来的能力	需要承担边界
目录 + URI	可浏览、可引用、可组合的上下文	组织质量与权限范围影响使用效果。
多层语义表示	先理解范围，再按需读取细节	摘要会失真或滞后，关键事实要回读。
全局检索 + 组装	把候选定位与任务上下文准备分开	命中列表与可用上下文是不同结果。
内容 / 索引分离	保留正本，支持派生索引重建	需要处理暂时不一致与部分失败。
Schema 记忆更新	提取、合并和引用有明确结构	模型判断仍需验证，不能保证事实正确。
宿主 / 核心分工	多客户端复用同一上下文服务	适配器的默认行为和版本需单独核对。

源码优先的版本校准

- ① 检索：类名 HierarchicalRetriever 容易造成误解；当前实现已经是全局召回和一次候选排序。目录树仍用于内容组织与摘要生成。[S07]
- ② Working Memory：升级文档描述默认关闭；当前代码 MemoryPolicy 的默认和缺省解析仍为 True。已落地的插件变化是 Codex resumeArchiveInject=false。不能据此宣称服务端默认不生成摘要。[S10][S14][S25]
- ③ 存储：向量库保存摘要等检索文本，“不存原文正本”不能简化成“不存任何内容”。路径锁与队列恢复也不应写成跨组件强事务。[S20][S21]

本报告结论只绑定封面提交。它是源码层面的系统设计分析，不代表对任意已安装服务配置、插件状态或生产数据进行过验证；遇到文档与代码差异时，以本次核验的代码为准。

APPENDIX A / SOURCE MAP

源码与文档索引 · 核心流程

点击 [Sxx] 可打开目标仓库固定提交中的文件。行号用于快速定位；引用不随 main 后续更新漂移。

[S01] 系统组装与基础设施生命周期

[openviking/service/core.py#L66](#)

[S02] 上下文类型与命名空间

[docs/zh/concepts/02-context-types.md](#)

[S03] L0 / L1 / L2 与目录语义文件

[docs/zh/concepts/03-context-layers.md](#)

[S04] VikingFS 的接口与组合结构

[openviking/storage/viking_fs/__init__.py](#)

[S05] 资源导入与内容更新计划

[openviking/utils/resource_processor.py#L303](#)

[S06] 解析、目录构建与语义生成

[docs/zh/concepts/06-extraction.md](#)

[S07] 全局检索与统一候选排序的实现

[openviking/retrieve/hierarchical_retriever.py#L93](#)

[S08] 上下文组装管线

[openviking/retrieve/context_assembler/pipeline.py](#)

[S09] 会话提交与归档的两阶段处理

[openviking/session/session.py#L1236](#)

[S10] 记忆策略实际默认值及解析

[openviking/session/memory_policy.py#L88](#)

[S11] 记忆提取与经验演化编排

[openviking/session/compressor_v3.py](#)

[S12] Schema 约束的记忆提取循环

[openviking/session/memory/extract_loop.py](#)

[S13] 记忆写回、合并与引用维护

[openviking/session/memory/memory_updater.py#L758](#)

[S14] Codex 插件默认配置

[examples/codex-memory-plugin/scripts/shared/config-schema.mjs#L160](#)

[S15] Codex 记忆插件及来源摘要

[examples/codex-memory-plugin/README.md](#)

APPENDIX B / SOURCE MAP

源码与文档索引 · 集成与治理

核心代码与概念文档交叉使用；S25 与 S10 的默认值差异已在第 7、14 页明确标注。

[S16] 上下文网关的边界与接入流程

[docs/zh/guides/15-context-gateway.md](#)

[S17] 上下文网关记忆内核

[context_gateway/kernel.py#L97](#)

[S18] 上下文编译与记忆整理路由

[openviking/service/compile_service.py](#)

[S19] 记忆整理的行为与范围

[docs/zh/context-compilation/06-memory-consolidation.md](#)

[S20] 双层存储与后端边界

[docs/zh/concepts/05-storage.md](#)

[S21] 路径锁、一致性与恢复边界

[docs/zh/concepts/09-transaction.md](#)

[S22] 租户、用户与 Peer 隔离

[docs/zh/concepts/11-multi-tenant.md](#)

[S23] 共享资源 ACL 与继承规则

[docs/zh/concepts/15-acl.md](#)

[S24] 任务与队列完成语义

[docs/zh/concepts/16-queue-lifecycle.md](#)

[S25] Working Memory 默认关闭的升级文档

[docs/zh/guides/working-memory-default-off.md](#)

[S26] 账号运行时配置管理

[openviking/config/manager.py#L187](#)

[S27] 数据加密及密钥边界

[docs/zh/concepts/10-encryption.md](#)

[S28] 检索 API 与关键词搜索约束

[docs/zh/api/06-retrieval.md](#)

[S29] 索引重建的持久化处理

[openviking/storage/queuefs/reindex_processor.py](#)

[S30] 服务端 MCP 接口

[openviking/server/mcp_endpoint.py](#)